A Fine-Tuned Transformer for Criterion-Level Clinical Trial Eligibility Screening with Traceable Evidence and Efficient Inference

Masooma Ali
Department of Computer Science
FAST NUCES, Islamabad
Roll No. 23I-0623 Course: Generative AI

Abstract—Matching a patient to a clinical trial requires judging the patient record against each inclusion and exclusion criterion individually and justifying every decision with evidence from the record. We fine-tune an open-weight decoder, Qwen2.5-1.5B-Instruct, with Low-Rank Adaptation (LoRA) on the TrialGPT Criterion-Annotations dataset so that it reads a patient note, a criterion type, and a criterion, and returns a structured eligibility label together with the identifiers of the supporting patient sentences. All four expert categories per criterion type are preserved, including not enough information and not applicable, and any verdict that cites no sentence is flagged for human review rather than treated as proof. Using a sealed, patient-grouped 37/8/8 split, fine-tuning raises held-out test macro-F1 from 0.234 (untrained base) to 0.560 and accuracy from 0.204 to 0.728. Beyond accuracy, we implement and validate three inference mechanisms from first principles against reference paths under greedy decoding: an explicit key/value (KV) cache that reproduces uncached tokens exactly while giving up to a 9.2× decode speed-up on long inputs; a tiled online-softmax attention kernel that matches a reference implementation to a maximum absolute error of 5×10^{-7}; and greedy speculative decoding that produces token-identical output at a 90–97% draft acceptance rate, yet runs 15–24% slower than plain greedy on our hardware—a slowdown we measure and explain. The system is delivered as a browser application that screens unseen notes, shows each verdict with its cited evidence, and exposes a live benchmark of the decoding mechanisms. It is an educational screening prototype, not a clinical enrollment tool.

Index Terms—clinical trial eligibility, clinical NLP, parameter-efficient fine-tuning, LoRA, KV cache, FlashAttention, online softmax, speculative decoding, evidence attribution.

I. Introduction and Problem Statement

Recruiting patients into clinical trials is slow and expensive, largely because screening a patient means reading a free-text record and checking it against a long list of inclusion and exclusion criteria, each of which may be met, not met, inapplicable, or impossible to decide from the available text. Large language models can automate parts of this reasoning [1], but a system that merely emits "eligible" or "ineligible" is not trustworthy for clinical use: a screening tool must say which criterion drove a decision and which sentence of the record supports it, and it must distinguish "the criterion is not satisfied" from "the record does not say."

This work builds such a system under three constraints. First, it must run on freely available hardware with open weights—no paid APIs or proprietary models. Second, it must preserve the full label taxonomy of the benchmark and attribute every decision to patient-record evidence. Third, beyond training a classifier, it must demonstrate a genuine, verified understanding of modern autoregressive inference: KV caching, efficient attention, and speculative decoding, each implemented and checked against a reference rather than switched on through a library flag.

Contributions. (1) A LoRA fine-tune of Qwen2.5-1.5B-Instruct for criterion-level eligibility with evidence-sentence attribution, evaluated on a sealed patient-grouped split with a documented before/after comparison. (2) From-scratch implementations of a cache-aware greedy decoder, a tiled online-softmax attention kernel, and greedy speculative decoding, each validated for exact token or numerical agreement with a reference and benchmarked. (3) A standalone browser application that performs criterion-level screening with traceable evidence, aggregates a preliminary patient-level verdict by explicit rules, and exposes a live inference-comparison view.

II. Related Work

LLMs for trial matching. TrialGPT [1] introduced the criterion-level annotation task and a prompting pipeline over GPT-4; we reuse its public annotations as the supervised benchmark but, as required, do not use its model-generated predictions or explanations as inputs or labels. Our contribution is a small, locally executable, fine-tuned model with verified inference internals rather than a closed-model prompting demonstration.

Transformer architecture and attention. The Transformer and scaled dot-product attention [2] underlie our target model. FlashAttention [3] shows that attention can be computed tile-by-tile without materialising the full score matrix, using the online softmax recurrence of Milakov and Gimelshein [4]; our educational kernel in Section III-F implements exactly this recurrence.

Parameter-efficient adaptation. LoRA [5] freezes the base weights and trains low-rank updates to selected projections, and QLoRA [6] adds quantisation; with only

TABLE I
Patient-grouped split (seed 42). The test split was not inspected during development.

Split	Patients	Rows (usable)	
Train	37	602	
Validation	8	206	
Test	8	206	
Total	53	1014	(+1 removed)

602 training rows, full fine-tuning would overfit and exceed our memory budget, so we adopt LoRA.

Speculative decoding. Leviathan et al. [7] and Chen et al. [8] accelerate decoding by drafting tokens with a small model and verifying them with the target in one pass, guaranteeing the target's exact greedy output. We implement the greedy variant and report where it does not pay off.

Model families. We compare Qwen2.5 [9] and SmolLM2 [10] as candidates and reject Qwen3 [11] for reasons given in Section III-C.

III. Materials and Methods

A. Dataset and Patient-Grouped Split

We use only the TrialGPT Criterion-Annotations dataset,[1] which contains 1,015 annotations over 53 patients: a patient description, a single criterion with its type (inclusion/exclusion), an expert eligibility label, and expert-identified evidence sentence IDs. To prevent leakage, we split by patient: unique patient IDs are sorted, shuffled with numpy.random.default_rng(42), and assigned 37/8/8 to train/validation/test. Every annotation for a patient stays in one split. One row with a blank criterion (annotation 883) has nothing to judge and is removed from training and scoring in all splits; the decision predates any results and is recorded in the published manifest. Table I summarises the split, which yields 602 usable training rows, 206 validation, and 206 test. The test split was sealed and read exactly once, for final evaluation.

B. Data Processing and Output Schema

The loader requests only the eight columns it needs; the gpt4_* and explanation_correctness columns are never read, so prior model outputs cannot leak into inputs or labels. Each patient note is a numbered list of sentences; we keep the original IDs so that predicted evidence is traceable to the exact source sentence, and an NLTK splitter assigns IDs to unseen notes in the application. The model is trained to emit a compact JSON object, {"label": <category>, "evidence": [<ids>]}. The label space is the dataset's own taxonomy: {included, not included, not enough information, not applicable} for inclusion criteria and {excluded, not excluded, not enough information, not applicable} for exclusion criteria. We deliberately keep not enough information and not applicable as first-class outcomes rather than collapsing to yes/no. Explanations are not learned: the only written explanations in the dataset are GPT-4's, which the rules bar; instead the application states the verdict in fixed wording and quotes the cited sentences verbatim, so an explanation cannot introduce an unsupported fact.

[1] https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations

TABLE II
Open-weight decoder candidates. All Apache-2.0.

Model	Params	Layers	Role
Qwen2.5-1.5B-Instruct	1543.7M	28	target (chosen)
Qwen2.5-0.5B-Instruct	494.0M	24	draft (chosen)
SmolLM2-1.7B-Instruct	1711.4M	24	candidate, rejected

Tokenisation and truncation. Inputs use the Qwen2.5 chat template. The application caps prompts at 2,048 tokens and refuses longer inputs with a message rather than silently truncating, because clipping a note could discard the very sentence that constitutes the evidence. No training row exceeded the limit, so no evidence was lost to truncation.

C. Model Selection

We require an open-weight decoder that (i) fits a 4 GB laptop GPU for the application and a free 16 GB Kaggle T4 for training, (ii) supports the autoregressive inference work, and (iii) has a draft sibling with an identical tokeniser for speculative decoding. Table II lists the candidates. We select Qwen2.5-1.5B-Instruct (Apache-2.0, 28 layers, 32k context) as the target and Qwen2.5-0.5B-Instruct (24 layers) as the draft; the two share an identical vocabulary of 151,665 tokens, special tokens and chat template, which is a precondition for speculative verification. SmolLM2-1.7B-Instruct is a reasonable alternative but did not fine-tune as well (Section III-D). We reject the Qwen3 family: its "thinking" mode is on by default and injects reasoning text that complicates structured JSON output, and the 4B variant needs ≈8 GB at float16, exceeding the laptop GPU.

D. Fine-Tuning with LoRA

We adapt the target with LoRA [5] on the attention projections (q,k,v,o_proj) at rank 32, α=64, dropout 0.05, training 8.72M parameters—0.56% of the 1.55B total. Optimisation uses AdamW, learning rate 2×10^{-4}, batch size 4 with 2 gradient-accumulation steps, 10% warmup, gradient clipping at 1.0, and gradient checkpointing. The objective is standard next-token cross-entropy over the JSON target. Decoding for all evaluation is greedy with repetition penalty fixed to 1.0; Qwen's bundled generation config enables sampling, which we override explicitly so that before/after comparisons are deterministic and token-equality with our custom decoders holds. Because the

mandatory split leaves only 8 "excluded" and 13 "not included" training rows, we optionally oversample rare labels in training by a power of 0.5. All hyperparameters and the final checkpoint were chosen on validation data only; early stopping used patience 2 on validation macro-F1. Training the selected configuration took ≈1,295 s on a T4 at a 7.55 GB peak.

E. KV-Cached Autoregressive Decoding

Autoregressive generation recomputes attention over all previous tokens at every step. We implement an explicit cache-aware decoder that, after a prefill pass over the prompt, stores each layer's key and value tensors and, for every new token, runs a single-token forward pass that appends one key/value row per layer and attends over the stored cache. Correct positional handling is essential: the new token's rotary position index is the running cache length, not 0. The causal mask is implicit, because a one-row query can only attend to keys that already exist. We also build an uncached reference decoder that reprocesses the whole growing sequence each step, giving an independent oracle for token-equality tests. Minor numerical differences between the two paths are expected and bounded (Section V-E) because the cached path changes the reduction order of floating-point attention.

F. Tiled Online-Softmax Attention

We implement reference scaled dot-product attention—$\mathrm{softmax}(QK^\top/\sqrt{d_k}+M)V$ with a numerically stable max-subtraction—and a tiled variant following FlashAttention [3]. The tiled kernel walks key/value blocks while maintaining, per query, a running maximum m, a running normaliser ℓ, and a running weighted sum, rescaling previous contributions by $e^{m_{\text{old}}-m_{\text{new}}}$ as the maximum grows [4]; it never materialises the full $N\times N$ score matrix. Under a causal mask, blocks that lie entirely in the future are skipped. Both are tested on small controlled inputs for agreement (Section V-F).

G. Speculative Decoding

Our greedy speculative decoder drafts k tokens with the 0.5B model, then runs one target forward pass over the drafted block to obtain the target's greedy token at each position. It accepts the longest prefix on which draft and target agree, always appends the target's own next token (so at least one token is committed per round even under full rejection), and rolls both models' KV caches back to the accepted length to keep state consistent. Because every committed token is the target's greedy choice, the output is provably identical to plain greedy target decoding, which we verify in Section V-G.

H. Application Workflow

The system is a FastAPI server that loads the model once and serves a React/Vite single-page interface (Fig. 1). A user pastes or uploads a note, reviews the auto-numbered sentences, enters or imports inclusion/exclusion criteria, and runs the screening; a single-GPU job queue processes one request at a time. Each criterion returns its predicted category, cited sentence IDs, extracted evidence text, and a fixed-wording explanation. A patient-level verdict is produced only as an explicit rule-based aggregation ("likely ineligible" if any criterion fails with cited evidence; otherwise "needs review" if anything is unknown, unverified, or uncited; otherwise "potentially eligible"), labelled preliminary, and never presented as a separately ground-truthed outcome. The interface exposes a live benchmark view (Fig. 6) that runs the three decoders and the two attention kernels on demand, and exports a self-contained HTML/JSON report carrying the model and adapter provenance. A banner states that the tool is an educational prototype.

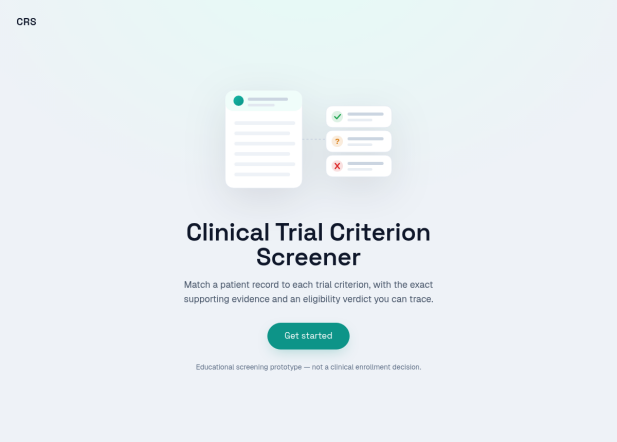


Fig. 1. Browser application: a patient record is matched to each trial criterion with traceable evidence. The prototype notice is shown throughout.

IV. Experimental Setup and Metrics

Hardware. Fine-tuning and full benchmarks ran on a free Kaggle Tesla T4 (16 GB). The application, tests, and small benchmarks ran on a laptop (i7-11800H, RTX 3050 Ti 4 GB); the deployment target in this report is a CPU-only machine on which the application also runs (slower, float32).

Metrics. For classification we report accuracy and macro-F1, per-class precision/recall/F1, and confusion matrices, overall and separately for inclusion vs. exclusion. For evidence retrieval we report sentence-level micro and macro precision/recall/F1; rows where both gold and predicted evidence are empty count as perfect ($\mathrm{F1}=1$), and the fraction of such rows is reported so the macro number is not misread. For inference we report prefill time, decode time, tokens/s, peak memory, draft acceptance rate, and token-equality against the reference, with warm-up discarded and medians over repeated runs.

TABLE III
Validation results across runs. os: oversample power, rk: LoRA rank. R3 (bold) is selected. All macro-F1 over the six scored labels.

Run	Model	rk	os	Acc.	Macro-F1	Ev-F1
R0	Qwen-1.5B (base)	16	–	0.175	0.147	0.106
R0b	SmolLM2 (base)	16	–	0.180	0.120	0.111
R1	Qwen-1.5B	16	0	0.752	0.534	0.462
R2	Qwen-1.5B	16	0.5	0.738	0.552	0.487
R3	Qwen-1.5B	32	0.5	0.743	0.558	0.509
R4	Qwen-1.5B (ev-first)	32	0.5	0.762	0.459	0.526
R5	SmolLM2	32	0.5	0.675	0.514	0.446
R6	Qwen-0.5B (draft)	32	0.5	0.738	0.446	0.491

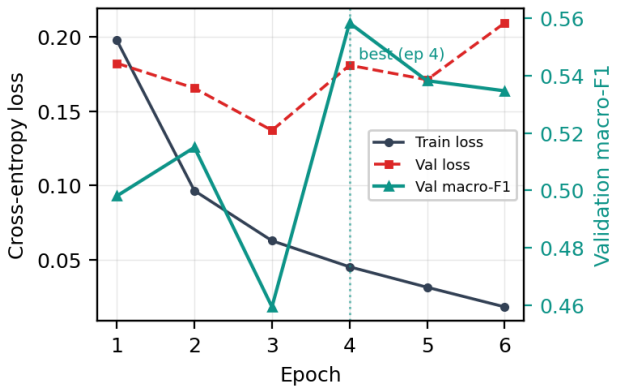


Fig. 2. Run R3 training. Validation loss and macro-F1 diverge after epoch 4 (dotted), the selected checkpoint.

V. Results and Discussion

A. Fine-Tuning and Before/After

Table III compares all runs on validation data. The untrained base (R0) scores 0.147 macro-F1; LoRA fine-tuning (R1–R4) more than triples this. Oversampling rare labels helps (R2 over R1), and rank 32 with oversampling (R3) gives the best validation macro-F1 (0.558) and the best evidence F1 among the Qwen runs, so R3 is selected by the pre-stated rule. Putting evidence before the label (R4) slightly raised evidence F1 but hurt label macro-F1 to 0.459, so the label-first format was kept. SmolLM2 (R5) and the 0.5B draft (R6) trained but scored lower, confirming the target choice. Fig. 2 shows R3's training: training loss falls monotonically while validation loss and macro-F1 diverge after epoch 4, which the patience-2 rule correctly selects as the checkpoint.

On the sealed test split, fine-tuning raises accuracy from 0.204 to 0.728 and macro-F1 from 0.234 to 0.560 (Table IV). A majority-label baseline reaches 0.626 accuracy but only 0.272 macro-F1, confirming that the accuracy gain is not merely from predicting the most common class. Validation and test numbers agree closely (0.558 vs. 0.560 macro-F1), indicating the configuration did not overfit the validation split.

TABLE IV
Before/after on identical held-out examples, greedy decoding.

Split	System	Accuracy	Macro-F1
Validation	Base (untrained)	0.175	0.147
Validation	LoRA (R3)	0.743	0.558
Test	Base (untrained)	0.204	0.234
Test	Majority-label	0.626	0.272
Test	LoRA (R3)	0.728	0.560

TABLE V
Per-class results on the test split (R3). "–": no support.

Class	Prec.	Recall	F1	Support
included	1.000	0.704	0.826	27
not included	0.500	0.143	0.222	7
not excluded	0.731	0.956	0.829	91
not enough information	0.667	0.606	0.635	66
not applicable	0.500	0.200	0.286	15
excluded	–	–	–	0

B. Criterion Classification

Table V gives per-class test results. The model is strong on the common verdicts—included (F1 0.826, precision 1.0) and not excluded (F1 0.829)—and on not enough information (F1 0.635). It is weak on the rare and subtle classes: not applicable (F1 0.286) and not included (F1 0.222, recall 1/7). The excluded class has no test examples, so its performance is unmeasurable on this split. Split by criterion type, inclusion accuracy is 0.773 (macro-F1 0.469) and exclusion accuracy 0.702 (macro-F1 0.427); the confusion matrix (Fig. 3) shows the dominant error is predicting not excluded / not enough information where the gold is a rarer decisive label.

C. Evidence Retrieval

Sentence-level evidence retrieval on the test split reaches micro-F1 0.523 (precision 0.577, recall 0.479). Of 206 rows, 136 have both gold and predicted evidence empty, 37 have both non-empty, 24 have gold evidence the model missed, and 9 have evidence the model added where the gold had none. Restricted to the 61 rows with gold evidence, macro-F1 is 0.528; for inclusion criteria specifically, evidence micro-F1 is 0.631. The large empty/empty count reflects that many "not excluded" verdicts are reached because a condition is simply never mentioned—which is precisely why the application never reads an empty citation as proof.

D. Error Analysis

A manual review of the 56 misclassified test rows (and 27 label-correct rows with different evidence) identifies two systematic habits. On exclusion criteria the model defaults to not excluded: 32 of 56 errors, learned from training data in which 74.7% of "not excluded" rows carry no cited sentence. On inclusion criteria it retreats

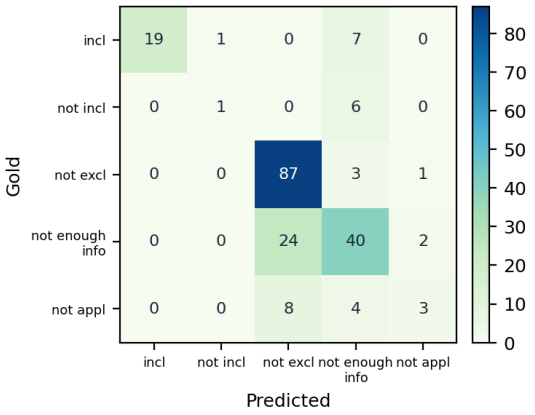


Fig. 3. Test confusion matrix (R3). Errors concentrate in the "not enough information" and "not excluded" columns.

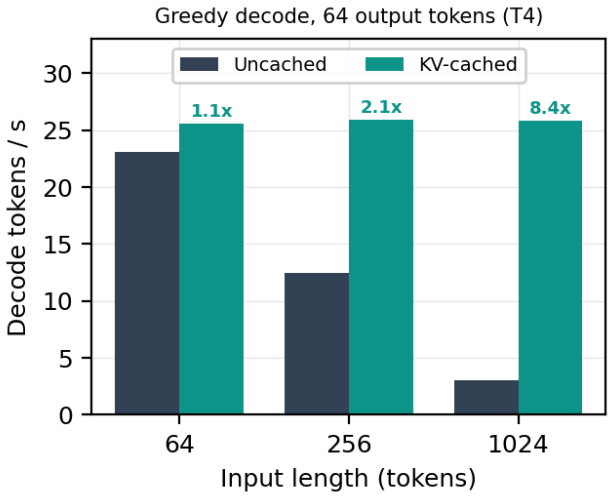


Fig. 4. KV-cache decode throughput on a T4. Labels show cached/uncached speed-up; output identical to the reference at every point.

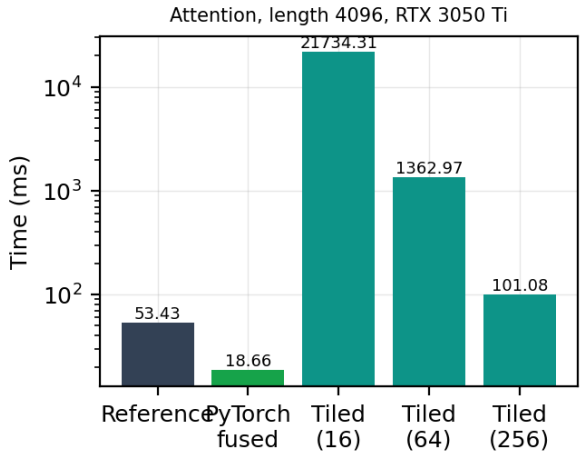


Fig. 5. Attention runtime (log scale), length 4096. The educational tiled kernel is correct but slower than the fused backend.

to not enough information when a clinical inference is needed, giving that class recall 1.0 but "not included" recall 0.14. A representative failure is a negated inclusion criterion—"no history of central venous catheterization"—where the note mentions arterial catheterization; the shared word triggers a polarity flip. Crucially, because that verdict cites no sentence, the application flags it and routes the patient to "needs review" rather than asserting ineligibility, so the user-facing failure mode is caution, not a false decision. The largest error cluster (17 rows) comes from one patient on one long trial, and the patient-level bootstrap interval for macro-F1 is wide (0.43–0.64) given only 8 test patients—results should be read with that uncertainty.

E. KV Cache Benchmark

Across input lengths of 64–1024 and output lengths of 16–256, the cached decoder produced token-identical output to the uncached reference at every setting, with a maximum absolute logit difference of 0.055 (mean 0.006), as expected from reordered floating-point reductions. Fig. 4 shows decode throughput: the uncached path slows sharply as the input grows (its per-step cost scales with sequence length), while the cached path holds ≈25.5 tokens/s. The speed-up therefore grows with input length, from 1.1× at 64 tokens to 8.4× at 1024 tokens (9.2× at 256 output tokens). This is the central practical benefit of KV caching and is exactly what the clinical setting needs, where prompts (note + criterion) are long.

F. Attention Benchmark

On controlled inputs with a causal mask, the tiled online-softmax kernel matched the reference to a maximum absolute error of 5×10^{-7} (mean 2.6×10^{-8}), confirming correctness, and correctly skipped future blocks (e.g. 6 of 16 tiles skipped at length 64, tile 16). On runtime (Fig. 5), however, the educational tiled kernel is slower than both the naive reference and PyTorch's fused backend—at length 4096 it takes several milliseconds versus a fraction of a millisecond for the fused path. This is expected: our kernel is pure Python/PyTorch and launches many small operations per tile, whereas FlashAttention's advantage [3] comes from a single fused CUDA kernel that keeps tiles in on-chip SRAM. The exercise demonstrates the algorithm's memory behaviour (no full score matrix), not a faster kernel.

G. Speculative Decoding Benchmark

With the fine-tuned 0.5B draft, speculative decoding produced output identical to greedy target decoding at every draft length $k \in \{2, 4, 8\}$, verifying correctness. Draft acceptance was high—0.97 at k=2, 0.90 at k=8—yet end-to-end throughput was lower than plain greedy: 0.76× at k=2 to 0.85× at k=8 (Table VI). The reason is

TABLE VI
Greedy speculative decoding vs. greedy target (T4, fine-tuned draft, 20 prompts). Output token-identical to greedy in all cases.

Mode	Tokens/s	Acceptance	Speed-up
Greedy target	23.9	–	1.00×
Speculative k=2	18.2	0.97	0.76×
Speculative k=4	19.3	0.91	0.81×
Speculative k=8	20.2	0.90	0.85×

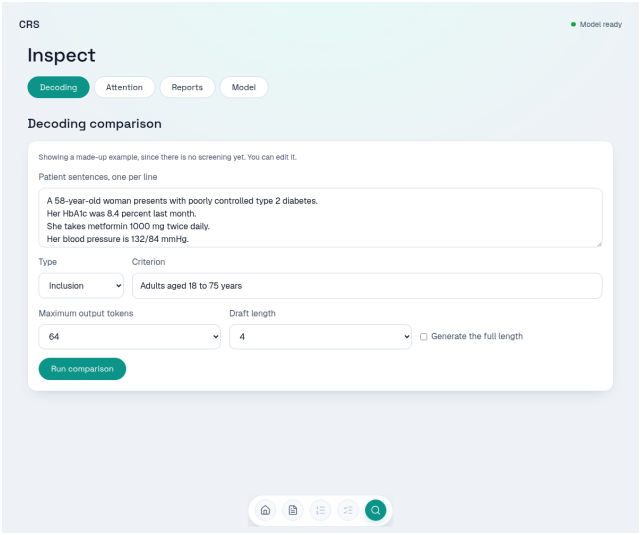


Fig. 6. In-application benchmark view: the three decoders run on demand and are compared on throughput, memory and token-equality.

the cost model: the target's single-token decode is already cheap on a T4, so the extra draft forward passes and the per-round verification overhead outweigh the tokens saved, especially on the short structured outputs this task produces. Speculative decoding pays off when the target step is expensive relative to the draft and outputs are long; neither holds strongly here. As the rubric anticipates, we report this slowdown honestly—the implementation is correct and the cache state is maintained; it simply does not accelerate this workload.

VI. Limitations

The mandatory split gives only 8 test patients, so metrics carry wide patient-level intervals and the excluded class cannot be measured at all. The model inherits the dataset's bias toward "not excluded"/"not enough information" and is weak on rare decisive labels. Evidence recall (0.48) means roughly half of gold citations are missed, though the application's flag-for-review behaviour mitigates the risk of over-confident decisions. Our attention and speculative implementations are educational: correct but not faster than optimised backends on this hardware. Finally, the system is a research screening prototype; it must not be used for real enrollment decisions.

VII. Conclusion

We presented a locally executable, LoRA-fine-tuned Transformer that screens patients against individual trial criteria, preserves the full expert label taxonomy, and attributes every verdict to patient-record sentences, lifting held-out macro-F1 from 0.234 to 0.560. We implemented and verified three inference mechanisms from first principles: a KV cache that is token-exact and up to 9.2× faster on long inputs, a tiled online-softmax attention kernel matching a reference to 5×10^{-7}, and greedy speculative decoding that is token-identical at 90–97% acceptance but slower on this workload—a result we measure and explain. The whole system is delivered as a browser application with traceable evidence and a live inference benchmark. Future work includes calibrated abstention on rare labels, better evidence recall, and evaluation on additional patient cohorts.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," Nature Communications, vol. 15, no. 1, p. 9074, 2024, arXiv:2307.15051.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017, arXiv:1706.03762.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Advances in Neural Information Processing Systems (NeurIPS), 2022, arXiv:2205.14135.

[4] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv preprint arXiv:1805.02867, 2018.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in International Conference on Learning Representations (ICLR), 2022, arXiv:2106.09685.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), 2023, arXiv:2305.14314.

[7] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in International Conference on Machine Learning (ICML), 2023, arXiv:2211.17192.

[8] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.

[9] Qwen Team, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[10] L. B. Allal, A. Lozhkov, E. Bakouch et al., "SmolLM2: When smol goes big – data-centric training of a small language model," arXiv preprint arXiv:2502.02737, 2025.

[11] Qwen Team, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.